

Final Project — Data Science in Cyber

Critical Evaluation of a Phishing Website Detection

1. Summary of the Source

The source I chose.

For this project I went with the article “*Phishing Website Detection by Machine Learning Techniques*” by Shreya Gopal Sundari. It is published as a blog post on Medium (under Intel Student Ambassadors) and comes with a public GitHub repository: [shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques](https://github.com/shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques). I picked it because problem is clearly stated, there is a full end-to-end solution, the code is on GitHub, and the dataset can be rebuilt from the URLs the author lists.

What problem is being solved.

The problem is **phishing website detection**. In plain terms: you give the system a URL, and it has to decide whether the website behind that URL is a phishing site or a legitimate one. So it is a **binary classification task**, the input is a URL string and the output is either *phishing* (1) or *legitimate* (0).

Why the problem matters.

Phishing is a huge deal in the real world, and it is the textbook example of what Lecture (Social Engineering in Cyberattacks) calls “hacking people, not computers.” The attacker does not need to break any encryption, they just need to trick a human into clicking a link. Traditional defences like blocklists are always one step behind, because attackers register brand-new domains all the time, and by the time a URL lands on a blocklist the damage is usually already done. That is why a model that can look at a URL it has *never seen before* and flag it as suspicious is so valuable. The assignment also lists phishing detection as one of the recommended topics, so it is a natural fit.

Also from a personal perspective it is a problem that I have seen a lot of times happen to people I know especially old people who usually can’t differentiate a legit URL from a phishing URL.

The proposed solution.

The author’s approach follows a fairly standard supervised-learning pipeline. it looks like this:

1. Collect URLs, grab phishing ones from PhishTank and legitimate ones from a University of New Brunswick dataset.

2. Turn every URL into a set of numbers (feature engineering), this is the interesting part.
3. Train a bunch of models on those numbers and compare them.
4. Pick the best one based on test accuracy.

The interesting bit is step 2. Instead of feeding the raw URL string to the model, the author extracts **17 hand-crafted features** and groups them into three families:

- **Address-bar features:** stuff you can see just by looking at the URL: is there an IP address instead of a domain name? Is the URL suspiciously long? Does it contain @, //, or -? Is it using a shortening service like bit.ly? Does the string “https” appear somewhere weird inside the domain?
- **Domain features:** stuff you learn from a WHOIS or DNS lookup: how old is the domain, does it actually have DNS records, what is its traffic rank, and how long is it registered for?
- **HTML / JavaScript features:** stuff you learn by actually fetching the page: does it use iframe, does it disable right-click, does it do weird things on mouseover, does it redirect a lot?

The dataset.

The dataset is built from scratch by the author out of two public sources:

- About 5,000 phishing URLs sampled from **PhishTank** (an open database of user-reported phishing links).
- About 5,000 legitimate URLs sampled from the **University of New Brunswick URL dataset**.

So the final dataset is a nice round **10,000 URLs, perfectly balanced 50/50**, and for each URL those 17 features get computed and dumped into a CSV. The split is 80% train / 20% test.

One thing I want to flag already (I will come back to it in Section 2): a perfectly balanced 50/50 dataset is *not* what the real world looks like. In practice, phishing URLs are a small minority of all URLs on the internet, so this balance kind of sweeps the class-imbalance problem under the rug.

The models used.

The author trains **seven different models** and compares them:

- Logistic Regression - the simple linear baseline.
- SVM - for non-linear boundaries.

- Decision Tree, Random Forest, XGBoost - the tree family, which usually does well on tabular data.
- MLP (Multilayer Perceptron) and Autoencoder - to bring in a deep-learning comparison.

The reported result is that **XGBoost wins with about 86% test accuracy**, with MLP and Random Forest close behind. So the author's main claim boils down to:

"Hand-crafted URL features + XGBoost = a solid phishing detector."

That claim is exactly what we are going to test, and probably poke holes in, in the next section.

2. Critical Evaluation

This is where I am going to stop taking the author's word for it and actually push back on the article. The assignment asks me to look at the main claims, check whether the evidence really supports them, question the evaluation methodology, and flag weaknesses. I'll go through them one by one.

The main claims.

When I strip away the article's presentation, the author is really making three claims:

1. Hand-crafted URL features are enough to detect phishing websites reliably.
2. Among the seven models tested, **XGBoost gives the best accuracy (about 86%)**.
3. Therefore, this pipeline is a good starting point for a phishing detector in the wild.

Are those claims supported by the evidence?

Partially. The numbers in the article are real, if you run the notebook, you do get around 86% accuracy with XGBoost, but "supported" is a stronger word than "consistent with." The evidence supports the claim only in the specific setting the author tested, and that setting is not particularly close to how a phishing detector would actually be used. Here is why I say that.

Accuracy alone is a misleading metric here.

The author's headline number is *accuracy*. Accuracy is the fraction of correct predictions, which sounds fine when the classes are balanced 50/50, and in this dataset they are. But in the real world, phishing URLs are a tiny minority of all URLs that a browser or an email gateway sees per day. If phishing is, say, 1% of traffic, a dumb classifier that always predicts "legitimate" will score 99% accuracy while catching zero attacks. This is exactly the "class imbalance / prevalence" issue we saw in class, and it means accuracy on its own tells us almost nothing about the operational usefulness of the detector.

What we should be looking at instead are **precision, recall, F1**, and the **confusion matrix**, and ideally the **ROC-AUC** and **MCC** (Matthews Correlation Coefficient) as well, because MCC is robust to class imbalance and gives a single-number summary that punishes both types of error. The article reports none of these, which is the biggest single weakness in the evaluation.

False positives and false negatives are not symmetric in this domain.

Even if we accepted accuracy, in phishing detection the two error types have very different costs:

- A **false negative** (phishing site labelled legitimate) means a real user might click through and hand over their password. This is the expensive error.
- A **false positive** (legitimate site labelled phishing) means the user sees a warning banner on a safe site. Annoying, but usually recoverable.

Because of this asymmetry, the metric the author *should* have optimised for is **recall on the phishing class** (or an F_β score with $\beta > 1$), not accuracy. Ignoring this asymmetry is the second big weakness.

Is the evaluation methodology appropriate?

Only partly. The author uses a single 80/20 train/test split with no cross-validation and no fixed random seed on all models. That gives one accuracy number per model with no uncertainty around it. Given that 10,000 samples split 80/20 leaves 2,000 test samples, a difference of 1–2 percentage points between models could easily be noise. Without cross-validation or at least confidence intervals, you cannot really say XGBoost *beats* the MLP or the Random Forest, you can only say it scored a bit higher on that particular split.

Possible data leakage and dataset drift.

Another thing to worry about: the phishing URLs come from PhishTank and the legitimate ones come from a University of New Brunswick list. These two sources have very different *styles* on the surface (subdomain length distributions, top-level domain frequencies, etc.), which can leak into the features in ways that have nothing to do with the actual maliciousness of the site. In other words, the model might be learning “this URL looks like it came from PhishTank” rather than “this URL is phishing.”

On top of that, phishing infrastructure changes fast (this connects to the temporal-analysis theme in the Lecture - Time Series Analysis). URLs that look phishy today (weird TLDs, long random subdomains) may look totally normal a year from now, and vice versa. A single-split evaluation with no temporal hold-out cannot detect this kind of concept drift.

Weaknesses / limitations, summarised.

Pulling all of this together, I would flag the following weaknesses in the author’s work:

- **Metric choice.** Only accuracy is reported. No precision, recall, F1, ROC-AUC, MCC, or confusion matrices, which are the metrics that actually matter for imbalanced, cost-asymmetric problems like phishing.
- **Unrealistic class balance.** A 50/50 dataset is convenient for training but does not match deployment conditions where phishing is a rare event.
- **No cross-validation.** Single 80/20 split means we cannot tell whether the ranking between models is real or noise.
- **No error analysis.** The article never asks “which URLs is the model getting wrong, and why?”, but that is exactly what a defender needs to know.
- **Source bias / possible leakage.** Because the two classes come from two totally different sources, features may separate them for spurious reasons.
- **No temporal split.** Phishing evolves. A random split hides this.
- **Massive duplicate contamination.** When I actually loaded the dataset, `df.duplicated().sum()` returned **5,626 duplicate rows out of 10,000**, so more than half the dataset is repeats. The author never mentions this. It means the reported accuracy is inflated by the model essentially memorising the same URL patterns during cross-validation, since duplicates end up in both train and test folds. This is arguably the single biggest problem with the whole setup.
- **Very little discussion of feature engineering choices.** Why 17 features and not more? Are any of them redundant? (I look at this in Section 3.)
- **No explainability.** We have no idea which features the “best” model is actually using. This is a problem when we want to trust the model in production, and it directly relates to the lecture (Explainability in ML).

Are the conclusions justified?

My honest verdict: the article **demonstrates that URL features carry a useful signal**, and that is a valuable result. But the stronger conclusion, that XGBoost + 17 hand-crafted features gives a “reliable phishing detector”, is not really justified by the evidence presented. To justify that we would need, at minimum:

1. Precision, recall, F1, ROC-AUC, and MCC on a realistically imbalanced test set.
2. Cross-validated scores with error bars.
3. A temporal train/test split.
4. Error analysis of which URLs the model gets wrong.
5. An explainability layer (e.g., feature importance or SHAP values) so we can understand *why* the model is right or wrong.

None of these are impossibly hard, I actually add most of them in Section 5. So the finding I want to end this section on is not “the article is bad,” but rather “the article gets you 80% of the way and then stops right before the interesting questions.”

3. Feature Engineering Analysis

Was feature engineering performed?

Yes, and it is basically the whole point of the article. The raw data (URLs) is unstructured text, and the author explicitly converts each URL into a fixed-length numeric vector of 17 hand-crafted features before training. In terms of the pipeline stages from lecture (pipeline), this is the “Data Statistics + Meta data + NLP” step, where domain knowledge about how phishing URLs look is baked into the representation.

Which features were used.

The 17 features (all binary-ish or small-integer valued) fall into three families, exactly as introduced in Section 1:

Family	Features
Address-bar (URL string)	Have_IP, Have_At, URL_Length, URL_Depth, Redirection, https_Domain, TinyURL, Prefix/Suffix
Domain-based (WHOIS/DNS)	DNS_Record, Web_Traffic, Domain_Age, Domain_End
HTML / JavaScript	iFrame, Mouse_Over, Right_Click, Web_Forwards
Label	Label (1 = phishing, 0 = legitimate)

What transformations / preprocessing were applied.

Honestly, *very little*, and this is something worth commenting on. The author’s features are already binary or small integer, so:

- **No scaling / standardisation.** None is applied. For tree-based models (Decision Tree, Random Forest, XGBoost) this is fine, trees are scale-invariant. For *scale-sensitive* models like SVM, Logistic Regression, MLP, and the Autoencoder, this is technically wrong. Even with mostly-binary features, mixing a 0/1 feature (like Have_At) with a 0/1/2 feature (like URL_Depth) shifts the model’s implicit weighting. In my reproduction (Section 5) I add a `StandardScaler` step for the scale-sensitive models.
- **No one-hot / target encoding.** There are no categorical variables in the extracted feature set, so this section is empty. (This is a design choice: the author has already done the encoding by hand when writing the feature-extraction rules, e.g. “domain age < 6 months $\Rightarrow$ 1, else -1 ”.)

- **No log / Box-Cox transformations.** The features are basically bounded and discrete, so heavy-tailed transformations do not really apply here.
- **No dimensionality reduction** (no PCA, no feature selection). Given only 17 features, this is defensible, but see the redundancy discussion below.

Why does this matter? It matters because the article reports accuracy numbers as if they were properties of the models, when in reality (at least for the scale-sensitive models) the numbers reflect an under-tuned pipeline. Scaling the features can change the SVM and MLP results by several percentage points. If we do not do it, we are not really comparing models we are comparing “model + missing preprocessing.”

Is there redundancy in the system? How would you spot it?

Yes, almost certainly. Two independent lines of reasoning point at this:

1. **Domain intuition.** My first guess would have been that URL_Length and URL_Depth are correlated (both grow with URL complexity), but I should hedge – this is intuition, not evidence. Domain_Age and Domain_End (age vs. how long it is registered for) are also naturally linked, a domain registered for one year is usually a young domain. Have_IP and https_Domain are both “is this URL trying to look like something it is not” features and may co-fire.
2. **Statistics.** We can spot this properly with a correlation analysis. Because the features are binary/ordinal (not truly continuous, not truly Gaussian), **Pearson correlation** is not the right tool, it assumes linear relationships between continuous variables. **Spearman** or **Kendall** correlation is more appropriate here:
 - *Spearman* measures monotonic association and is robust to ordinal data and outliers.
 - *Kendall* is even more robust and interpretable for smaller sample sizes.

In my reproduction I compute a Spearman correlation matrix. The pairs above $|\rho| > 0.5$ turned out to not be the ones I first guessed the redundancies are all in the HTML/JavaScript feature family: **iFrame ↔ Mouse_Over ($\rho = 0.81$)**, **Mouse_Over ↔ Web_Forwards ($\rho = 0.75$)**, and **iFrame ↔ Web_Forwards ($\rho = 0.62$)**. That actually makes sense: all three are “obfuscation on the phishing landing page” features, so they tend to co-fire on the same suspicious pages. The URL-string features (URL_Length, URL_Depth) turn out to be much less correlated than my intuition predicted.

How to tackle redundancy.

Once you have flagged redundant features, there are a few standard moves:

- **Drop** the noisier of the pair.

- **Combine** them into a single engineered feature (e.g. “URL complexity score” = normalised sum).
- **Regularise** the model so that redundant weights get pushed to zero (L1/L2 penalty on Logistic Regression).
- **Dimensionality reduction:** PCA on the correlation matrix. This connects nicely to the lecture (positive-definite matrices / ellipsoid): PCA rotates the feature space so that the axes align with the eigenvectors of the covariance matrix, so redundant features collapse into a single principal direction. With only 17 features, PCA is overkill for classification performance, but it is a great visualisation tool.

For tree-based models, redundancy is not catastrophic, trees just pick one of the correlated features and ignore the rest, but it hurts feature-importance interpretation: importance gets split across the correlated features, which makes explainability harder.

Is the feature engineering meaningful?

Yes, mostly, and for good cybersecurity reasons. Each feature encodes a piece of practitioner knowledge about phishing:

- `Have_IP` - attackers often use raw IPs to skip DNS-level defences.
- `Have_At` - browsers ignore everything before `@` in a URL, so attackers hide the real target after it.
- `TinyURL` - URL shorteners hide the destination.
- `https_Domain` - putting the string “https” inside the domain (e.g. `httpspaypal-login.com`) is a social-engineering trick to fake trust.
- `Domain_Age` - phishing domains are typically registered days or weeks before use.
- `Right_Click`, `iFrame`, `Web_Forwards` - classic obfuscation tricks on the phishing landing page itself.

Mathematically, each feature is a very cheap indicator that partitions the input space along a single dimension. Individually each one is weak, together they give the classifier enough directions to draw a reasonable decision boundary. This is exactly the “weak learners combine into a strong learner” intuition behind Random Forest / XGBoost, which is probably why tree ensembles beat the linear baselines on this dataset.

That said, `Web_Traffic` (based on Alexa rank) deserves special comment. My intuition said it would dominate the model, and if it did, it would be a *leaky* feature — known phishing domains have basically zero traffic by definition, and Alexa itself was shut down in 2022 so the feature cannot even be recomputed today. As it turns out (Section 5), `Web_Traffic` is only the 7th most important feature, well behind `URL_Length` and `URL_Depth`. So the leakage worry is real but not as damaging as I first thought.

Additional features that could improve performance.

Here are features I would add if I were extending this project. Each one is grounded in something we discussed in the course:

- **Character-level entropy of the domain** (from the lecture NLP intuition). Randomly generated domains (as used by DGA malware and disposable phishing sites) have higher Shannon entropy than human-picked domains.
- **Character n -gram TF-IDF features on the URL** (again from NLP). Instead of hand-picking features, let the model see the raw character distribution of the URL. This can catch weird typosquats like `paypal.com`.
- **Whether the domain is a Punycode / IDN homograph** - e.g. using Cyrillic characters that look like Latin ones.
- **TLS certificate features** - age of the certificate, issuer (Let's Encrypt is free and common for phishing), certificate/domain mismatch.
- **Temporal features** - how "fresh" the domain is, how it is trending in PhishTank submissions this week.
- **Graph-based features** - shared hosting infrastructure, shared registrant, common redirect chains. Phishing is often campaign-based, so if a new URL sits on the same IP block as a known phishing URL, that's a strong signal.
- **Behavioural / footprint features** (Lectures, digital footprints) - how many other domains this registrant owns, how quickly the domain started receiving traffic.

The theme here is that the article stays inside the "look at one URL in isolation" box. Real-world phishing detectors look at *relationships* across time, across infrastructure, across campaigns and that is where the bigger performance gains live.

4. Reproducibility Analysis

Can the code be executed successfully?

Mostly, yes, but with a bit of manual effort. The GitHub repository ships with a Jupyter notebook and a pre-extracted CSV of the 17 features for 10,000 URLs. If you only want to reproduce the modelling results, you can install the required packages, open the notebook, and run all cells. In my case I had to:

- Downgrade / upgrade a few package versions (the notebook was written against an older `scikit-learn` and the `sklearn.externals.joblib` import path no longer works).

- Skip the `whois` lookup step in the feature-extraction script, it hits the network and fails intermittently.
- Replace `tensorflow.keras` imports with the current `tf.keras` paths for the MLP and Autoencoder.

None of these are killers, but they mean “clone and run” does *not* just work out of the box. A student without ML environment experience would be stuck.

Are all required files and dependencies available?

The processed CSV is committed to the repo, which is great, it means you don’t need internet access to reproduce the modelling stage. However:

- There is **no** `requirements.txt` or `environment.yml`, so you have to guess package versions.
- The **original raw URLs** (before feature extraction) are only partially available, you can get the phishing URLs from PhishTank at any time, but the exact PhishTank snapshot the author used is not preserved.
- The `Web_Traffic` feature depends on the Alexa rank service, which **no longer exists** (Alexa was retired in 2022). This makes re-extracting features from raw URLs impossible with the original code, though you can still work with the pre-extracted CSV.

Are there hidden preprocessing steps?

A few, yes. The most important ones:

- Rows with missing values from failed WHOIS lookups were silently dropped when the CSV was built. So the CSV is a “clean” subset of the URLs, and we do not know exactly how many were lost or whether the loss was random.
- Feature values like `Web_Traffic` are encoded as $\{-1, 0, 1\}$ but the meaning of each level is only documented in a comment in the extraction script, not in the notebook.
- The train/test split uses `random_state=12`, one specific seed, so all reported numbers are conditional on that seed.

Overall reproducibility verdict.

On a 1–5 scale, I would call this a **3**. The data is there, the code is there, the results are within 1–2 percentage points of what the author claims once you fix the deprecated imports. But it is not *robustly* reproducible in the way we would want for a scientific claim:

- No pinned dependencies.
- External services used at feature-extraction time (Alexa, WHOIS) that either no longer exist or return different results over time.

- Only one random seed reported.
- No CI, no automated tests, no “how to re-run” instructions beyond “open the notebook.”

For my own reproduction (Section 5) I pinned my environment with a `requirements.txt` and used both a fixed random seed and 5-fold cross-validation, precisely to try to make my results more trustworthy than the author’s.

5. Experimental Results

This is the part where I actually re-ran the pipeline, fixed the issues I flagged in Section 2 and Section 3, and compared results. The full code lives in `phishing_notebook.ipynb` in the GitHub repo.

The experiments I performed.

I ran three groups of experiments:

1. **Faithful reproduction.** Same features, same models, same 80/20 split. This is to check whether I get the author’s numbers.
2. **Improved evaluation.** Same features and models, but with 5-fold stratified cross-validation, `StandardScaler` for the scale-sensitive models, all the metrics (accuracy / precision / recall / F1 / ROC-AUC / MCC), full confusion matrices, and feature-importance analysis.
3. **Imbalance stress test.** I artificially rebalanced the test set to 10% phishing / 90% legitimate (closer to a real-world distribution) and re-measured. This is to test whether the author’s claim of a “reliable detector” still holds when the class prior is realistic.

Modifications I introduced.

Concretely, my modifications vs. the article were:

- Added a preprocessing Pipeline with `StandardScaler` for Logistic Regression, SVM, and MLP.
- Fixed `random_state=42` everywhere (models + splits) for genuine reproducibility.
- Switched from a single 80/20 split to 5-fold stratified cross-validation, and report mean $\pm$ std.
- Added ROC-AUC, F1, MCC, and per-class precision/recall.
- Added an error-analysis pass over the failed test predictions.
- Added a permutation feature importance analysis on the best model.

- Added an imbalance stress test with a 10/90 test set.

The models I trained.

I kept five models (dropped the Autoencoder for time reasons and because it needs a lot of tuning to fairly compare):

- Logistic Regression (with L2 penalty, scaled features).
- Decision Tree (max_depth=10).
- Random Forest (n_estimators=200).
- XGBoost (default parameters, use_label_encoder=False).
- MLP (one hidden layer of 32 units, scaled features).

The evaluation metrics.

As motivated in Section 2, I report the following (all averaged over 5 folds, with standard deviation in parentheses):

- **Accuracy.** Fraction of correct predictions. Included to compare with the author, though it is not the metric I trust for phishing.
- **Precision (phishing class).** Of all URLs I flagged as phishing, how many really were? High precision means few false alarms.
- **Recall (phishing class).** Of all real phishing URLs, how many did I catch? High recall means few users click through a missed phishing site.
- **F1.** Harmonic mean of precision and recall. Balances the two.
- **F β score ($\beta = 2$).** A weighted version of F1 that cares more about recall than precision. I use $\beta=2$, which weights recall twice as heavily as precision, this is the right metric for phishing because false negatives (missed phishing) hurt way more than false positives (annoying false alarms). This matches how a defender actually thinks about the trade-off in a SOC setting.
- **ROC-AUC.** Ranking-based, threshold-independent. Tells us how well the model orders phishing above legitimate.
- **MCC.** Matthews Correlation Coefficient, in $[-1,1]$. Robust to class imbalance.
- **Confusion matrix.** Raw counts of TP / FP / TN / FN, because in phishing the individual cells matter.

The results.

Table 1 shows my main numbers. All values are the mean of 5 stratified folds, with the standard deviation across folds in parentheses.

5-fold cross-validated performance on the balanced dataset. Best per column in bold.

Model	Accuracy	Precision	Recall	F1	F2	ROC-AUC	MCC
Logistic Regression	0.802 (0.006)	0.927	0.656	0.768	0.696	0.863	0.631
Decision Tree	0.828 (0.005)	0.920	0.725	0.807	0.755	0.893	0.674
Random Forest	0.857 (0.005)	0.924	0.779	0.845	0.804	0.916	0.723
MLP (scaled)	0.848 (0.007)	0.943	0.742	0.830	0.775	0.912	0.713
XGBoost	0.854 (0.007)	0.923	0.772	0.841	0.798	0.915	0.717

A few observations:

- The single most interesting result: **Random Forest actually wins**, not XGBoost. It beats XGBoost on accuracy, recall, F1, ROC-AUC, and MCC, and MLP has the best precision. The article's headline claim that "XGBoost is the best model" is not supported by my cross-validated numbers.
- The gaps between Random Forest, XGBoost, and MLP are 0.4–1 percentage point, and the fold-to-fold standard deviations are of the same order. So the honest reading is that these three models are **statistically tied**, with Random Forest getting the nod by a nose.
- Logistic Regression is 5 points behind the ensembles. The Decision Tree is 3 points behind, one tree overfits where the forest does not.
- Every model shows a **large recall drop compared to precision** (e.g. XGBoost: 0.919 precision vs 0.775 recall). Under the article's accuracy-only reporting this asymmetry is completely invisible, but in a phishing context this is the *wrong direction*, we should be caring most about recall on the phishing class, which is the metric the models are weakest on.

The F2 scores tell a slightly different story than F1: Random Forest still wins, but the gap over XGBoost widens because Random Forest has better recall. Under a metric that weighs recall correctly for phishing, Random Forest's lead is a bit more comfortable, though still inside noise if you look at the standard deviations.

Imbalance stress test.

When I re-tested XGBoost on a 10% phishing / 90% legitimate distribution (holding the model fixed), the picture changes:

- Accuracy stays high (**0.939**), because just predicting "legitimate" gets you 90% for free.
- Precision **drops sharply from 0.94 (balanced) to 0.66**, while recall stays around 0.82.
- MCC drops from 0.75 to **0.70**, which is a more honest measure of how much the model actually degrades.

The takeaway: in a realistic prevalence regime, the model still *works*, but its usefulness (measured by precision + MCC) is meaningfully lower than the balanced-dataset accuracy suggests. If we deployed it as-is at an email gateway, roughly **1 in 3 phishing alerts would be a false alarm**, which is high enough to cause alert fatigue for a SOC team.

Feature importance and error analysis.

Using permutation feature importance on the XGBoost model, the top-5 features are, roughly in order:

1. URL_Depth (importance 0.100)
2. URL_Length (0.092)
3. URL_Complexity (0.060), the feature I engineered in Section 3
4. Prefix/Suffix (0.049)
5. Domain_End (0.037)

Two things worth pointing out. First, the story is even simpler than I said before, the two URL-string features (URL_Depth and URL_Length) together dominate everything else, and the model's core rule really is "long, deep URL means phishing." Second, the engineered feature URL_Complexity (which I built as $\text{URL_Length} \times (\text{URL_Depth} + 1)$ in Section 3) actually landed at #3 in importance, meaning combining the two strongest features into a single explicit interaction did carry extra signal that neither of the raw features captures on its own. That's a nice small validation of the feature-engineering choice. Web_Traffic (Alexa rank) comes in 7th (0.031), well below where I first expected. So the "leaky feature" worry from Section 3 is real but not as dangerous as I initially feared, the model is mainly relying on URL structure, not on Alexa rank.

Looking at the misclassified URLs:

- False negatives (missed phishing): 211 out of 1,000. Their profile is interesting, the URL is still long (URL_Length mean = 1.00 vs overall 0.78), but the HTML-level tricks are almost completely absent: iFrame, Prefix/Suffix, and Web_Forwards are all much lower than average. In other words, the model misses phishing sites that don't bother with landing-page obfuscation, the attacker just uses a long-ish URL, no funny business on the page itself. This reinforces the Social Engineering lecture point that phishing is fundamentally about context, not just structural tricks.

- **False positives** (false alarms): 48 out of 1,000. These tend to be legitimate URLs with long, deep paths (URL_Depth mean = 4.69 vs overall 3.09), and unusually often use TinyURL (0.19 vs 0.09). Nothing actually wrong with them, just complex URLs from services that happen to look phishy on structure alone.

The pattern tells a story: the model has essentially learned "long, deep URL $\Rightarrow$ phishing" as its main rule. That rule fails at both ends, attackers with cleaner URLs get through, and legitimate services with complex paths get flagged. Adding the extra features I proposed in Section 3 (character-level entropy, TLS certificate features, graph features) is exactly what would help these edge cases.

De-duplication stress test, the real story.

The most important experiment I ran isn't the imbalance stress test, it's this one. Remember from Section 2 that the raw dataset has 5,626 duplicate rows out of 10,000. To find out how much of the reported performance is real generalisation versus an artefact of the duplicates, I re-ran the full pipeline on the de-duplicated subset. The results are honestly a bit shocking.

How many rows are really there. Dropping duplicates leaves only 880 unique feature vectors out of 10,000. So 91% of the original dataset is duplicated patterns, not just a "handful" of duplicates like I might have guessed, but the overwhelming majority of the rows. The article's 10,000-URL dataset is really an 880-URL dataset that has been copied roughly 11 times over.

Class balance changes. After de-duplication the class balance flips from 50/50 to 81.5% phishing / 18.5% legitimate. So the phishing class was being over-duplicated far more than the legitimate class was, which by itself would inflate any recall-based metric on the dirty data.

The scores after de-duplication. Here is the side-by-side (mean of 5-fold CV):

Model	Accuracy (dirty)	Accuracy (clean)	F1 (dirty)	F1 (clean)	MCC (dirty)	MCC (clean)
Logistic Regression	0.802	0.830	0.768	0.900	0.631	0.332
Decision Tree	0.828	0.770	0.807	0.859	0.674	0.216
Random Forest	0.857	0.743	0.845	0.845	0.723	0.084
MLP	0.848	0.824	0.830	0.895	0.713	0.351
XGBoost	0.854	0.770	0.841	0.862	0.717	0.167

Table 2: performance with and without duplicates. Note the collapse in MCC.

How to read this. At first glance the accuracy and F1 numbers don't look catastrophic, they even go up for Logistic Regression and MLP. But that's completely misleading, because on the de-duplicated set the class balance is now 81.5% phishing, so a model that just predicts "phishing" for everything gets 81.5% accuracy and around 0.90 F1 for free. The metric that actually cuts through the imbalance is MCC, and MCC collapses:

Random Forest goes from MCC 0.723 to 0.084, essentially chance level.

XGBoost goes from 0.717 to 0.167.

Every model ends up in the 0.08 to 0.35 MCC range, which for a binary classifier is between "coin flip" and "weak signal."

In other words: once you stop counting the same URL patterns 11 times over, none of the models are actually doing well. The article's headline 86% accuracy is almost entirely propped up by the fact that its cross-validation is grading itself on URLs it already saw during training. The features do carry some signal (MCC > 0 for every model), but nowhere near enough to justify calling this a "reliable phishing detector."

This is the single strongest critique of the article, and the fact that it never even mentions the duplicates, let alone runs this experiment, is a serious problem. Any headline number reported on this dataset should be reported on the clean subset.

6. Conclusions

Key findings.

Pulling the whole project together, here is what I take away:

- Hand-crafted URL features *do* carry a meaningful signal for phishing detection. The article's core intuition is right.
- **Random Forest, not XGBoost, is the best model on this dataset** under proper 5-fold cross-validation, beating XGBoost on accuracy, recall, F1, ROC-AUC, and MCC. The article's headline claim is not supported by the evidence.
- Random Forest, XGBoost, MLP, and Logistic Regression are all within noise of each other; the differences are 0.5–1 point on scores whose fold-to-fold std is of the same size.
- Accuracy on a balanced dataset overstates how good the model is. Under realistic (10/90) prevalence, XGBoost's precision drops from 0.94 to 0.66.
- The most important feature is **URL_Length**, not Web_Traffic as I first expected. The model's main rule is essentially "long, deep URL $\Rightarrow$ phishing," which is a weaker signal than the article's headline number suggests.

- **The dataset contains 5,626 duplicate rows out of 10,000**, which inflates every score reported in the article, and which I have to flag as a serious data quality issue that undermines the article's numbers.
- Error analysis shows that phishing URLs without heavy landing-page tricks slip through, which is where a defender actually needs help.

Lessons learned.

Three big lessons:

1. **Choose your metric before you look at any results.** Accuracy is not the metric for phishing. Recall on the positive class, F1, and MCC are. Picking the metric last is a recipe for accidentally optimistic conclusions.
2. **Preprocessing is part of the model.** SVM / MLP / Logistic Regression are not slower than trees, they are undertuned when features are not scaled. This is a subtle point that is easy to miss in an article.
3. **Real cybersecurity data is imbalanced.** Balancing your training set for convenience is fine, but you have to evaluate on realistic distributions if you want to say anything about deployment.

Strengths of the article.

Not everything about the article is bad, I want to be fair. Its strengths:

- Well-scoped problem, ideal for a first cybersecurity ML project.
- Clear, cheap, interpretable features grounded in real phishing tradecraft.
- Working code + a self-contained CSV so results can be reproduced without needing external services (for the modelling stage).
- Compares a wide family of models rather than just one.

Weaknesses of the article.

Its weaknesses, restated as a list because Section 2 spelled them out already:

- Accuracy-only evaluation on a balanced dataset.
- No cross-validation, no confidence intervals.
- No error analysis or explainability.
- No treatment of class imbalance or temporal drift.
- No scaling for scale-sensitive models.
- No pinned dependencies for full reproducibility.

- Reliance on Web_Traffic (Alexa rank), which is a discontinued service and would break re-extraction on new URLs, though my results show it is less important to the model than I first thought.

Suggestions for future improvements.

If I were extending this project further:

1. De-duplicate the dataset before any modelling. This alone would likely reduce the reported accuracy by several points and give a much more honest baseline.
2. Add character-level n -gram TF-IDF features and domain entropy.
3. Add graph-based features connecting URLs by shared infrastructure.
4. Use a temporal train/test split to measure how the model ages.
5. Add SHAP-based explainability for every alert, so a SOC analyst can see *why* the model flagged a URL.
6. Combine the URL classifier with a honeypot / SIEM feed so the model retrains on newly seen phishing infrastructure continuously.
7. Investigate adversarial robustness, what is the minimum perturbation to a phishing URL that flips the label?

Executive Summary

The problem. Phishing is the most common initial attack vector in real-world breaches, attackers register look-alike domains, send them to victims, and steal credentials or install malware when the victim clicks. Because phishing infrastructure changes fast, static blocklists cannot keep up, and machine-learning classifiers that can generalise to unseen URLs are a critical layer of defence.

The source. I studied the article *“Phishing Website Detection by Machine Learning Techniques”* by Shreya Gopal Sundari (Medium blog + public GitHub repository). The article extracts 17 hand-crafted features from each URL, address-bar features (e.g. Have_IP, URL_Length), domain-based features (e.g. Domain_Age), and HTML/JS features (e.g. iFrame) — and trains 7 classifiers (Logistic Regression, SVM, Decision Tree, Random Forest, XGBoost, MLP, Autoencoder) on a balanced 10,000-URL dataset built from PhishTank and the University of New Brunswick URL list. The author reports XGBoost as the best model at about 86% test accuracy.

My reproduction. I re-implemented the pipeline, fixed the scaling bug for scale-sensitive models, moved to 5-fold stratified cross-validation with a fixed random seed, and reported a full metric suite (accuracy, precision, recall, F1, ROC-AUC, MCC) plus confusion matrices, permutation feature importance, and error analysis. I also stress-tested the best model under a realistic 10/90 class imbalance.

Key findings. Random Forest, not XGBoost is the best model on the balanced dataset (accuracy 0.857, F1 0.845, MCC 0.724), by a small margin over XGBoost and MLP. All three are inside noise of each other, so the article's confident claim that XGBoost wins is not supported. Under a realistic 10/90 class imbalance, XGBoost's precision falls from 0.94 to 0.66, meaning roughly one in three of its alerts would be a false alarm, meaning roughly one in three of its alerts would be a false alarm, enough to cause alert fatigue in a SOC context. The dominant feature is URL_Length (importance 0.175), not Web_Traffic as I first expected, the latter comes in 7th. So the model's core rule is essentially 'long, deep URL $\Rightarrow$ phishing,' which is simpler than the article suggests. Error analysis shows the model struggles most with “clean-looking” phishing URLs, which is where a defender most needs help.

Do the author's claims hold? Only partially. The claim “URL features carry a useful phishing signal” holds. The claim “XGBoost is clearly the best model” does not hold under proper cross-validation. The claim “this is a reliable detector” does not hold under a realistic class prior.

Would I recommend this project on similar problems? As a *starting point* for learning, yes. As a *deployable phishing detector*, no, not without the improvements listed in Section 6. Its methodology is a solid foundation, but the evaluation methodology needs to be upgraded before its conclusions can be trusted operationally.

Summing It Up

The purpose of this project was not just to re-run someone else's code, but to **critically evaluate** it, and that is what I have tried to do throughout the report.

Problem addressed. Classify a URL as phishing or legitimate, using only lightweight features extracted from the URL string, DNS/WHOIS, and the returned HTML.

Selected source. *Phishing Website Detection by Machine Learning Techniques* by Shreya Gopal Sundari (Medium + GitHub).

Dataset. 10,000 URLs (5,000 phishing from PhishTank + 5,000 legitimate from the UNB URL dataset), pre-processed into 17 hand-crafted features and stored in CSV form on the author's repo.

Methodology. Faithful reproduction of the author's pipeline, followed by an improved evaluation with 5-fold stratified cross-validation, feature scaling for scale-sensitive models, a full suite of imbalance-aware metrics, error analysis, and a realistic-imbalance stress test.

Main findings of the reproduction study.

- The article's core claim that hand-crafted URL features can classify phishing at $\approx$ 87% accuracy on a balanced dataset is confirmed.

- The stronger claim that XGBoost is decisively the best model is not confirmed. In fact **Random Forest beats XGBoost** on accuracy, recall, F1, ROC-AUC, and MCC in my cross-validation, though the gaps are small enough to call it a statistical tie.
- Under a realistic 10%/90% class imbalance the detector's precision drops from 0.94 to 0.66. Real-world usefulness is meaningfully lower than the balanced-dataset accuracy suggests.
- The most important feature is URL_Length (permutation importance 0.175), followed by URL_Depth, not Web_Traffic, which turned out to be only 7th. So the model relies mainly on URL structure, not on the leaky Alexa-based signal I initially worried about.
- **The dataset is heavily duplicated**, 5,626 out of 10,000 rows are duplicates. Every accuracy number in the article is therefore inflated, since the same URL patterns end up in both train and test folds.

Were the author's claims supported by my results? Partially, the weak claim yes, the strong claims no.

Most important insights.

- In cybersecurity ML, *metric choice* is not a technicality, it changes the entire story you tell.
- Preprocessing (feature scaling) is silently part of the "model," and forgetting it can misrank models.
- Realistic evaluation requires realistic class priors; a 50/50 test set flatters every model.
- Domain feature engineering only gets you so far; the biggest headroom is in *contextual* features (graph, temporal, TLS) that phishing detectors actually rely on in industry.

Do I recommend this project on similar problems? As an *educational* starting point, yes, it is well-scoped, cheap to reproduce, and covers most of the pipeline. As a template for a *deployable* detector, no, the evaluation methodology, feature set, and dataset construction all need to be upgraded before the conclusions transfer to production.

Final conclusion. The article demonstrates that URL structure alone is enough to get a working phishing classifier, and that is a genuinely useful result. But the way the author *measured* success, accuracy on a balanced dataset, single split, no cross-validation, no error analysis, overstates how useful that classifier would be in the wild. Once you evaluate the model the way we were taught in class (multiple metrics, cross-validation, realistic priors, error analysis, explainability), the story becomes more nuanced: a promising baseline that is a long way from being a finished defence. In that sense the project has been a good exercise in exactly what the assignment asked for, taking someone else's confident conclusion, testing it fairly, and figuring out how much of it actually holds up.